Архітектура системи: Клон Booking.com 3D

Модульна клієнт-серверна платформа • Django 5 + DRF • 3D Інтер'єри 3ds Max

Booking.com 3D MVP

Django REST API + 3D Max

🏛️ 1. Загальна концепція та підсистеми платформи

Клієнтські інтерфейси (SPA)

React 18 + TS + Vite

- **Клієнтський портал:** пошуковий рушій, каталог житла, фільтри зручностей, сторінка помешкання.
- **3D-Галерея інтер'єру:** інтерактивний перемикач кімнат та ракурсів з 3ds Max.
- **Кабінет власника & Адмінка:** управління житлом, цінами + **Django Admin (/admin/)**.

Серверна частина & Сховище

Python 5 + Django DRF

- **Backend REST API:** Django 5 + Django REST Framework (DRF), SimpleJWT, Swagger (drf-spectacular).
- **База даних:** PostgreSQL 16 (11 моделей) через Django ORM.
- **Сховище медіа:** оптимізовані рендери WebP/JPG (< 300KB) + Django Media Storage.

👥 2. Ролі користувачів та матриця доступу (RBAC)

РОЛЬ	АВТОРИЗАЦІЯ	ДОЗВОЛЕНИЙ ФУНКЦІОНАЛ	БЕЗПЕКА
Гість (Guest)	Анонімний	Пошук за локацією/датами, фільтри, перегляд фото та 3D-галереї інтер'єрів.	AllowAny (Публічні GET)
Користувач (User)	SimpleJWT	Створення та скасування бронювань, список "Мої бронювання", збереження в "Обране", відгуки.	IsAuthenticated (Bearer JWT)
Власник (Owner)	JWT + Role Owner	Додавання/редагування власного житла, завантаження фото й 3D-рендерів, керування ціною та календарем.	IsOwnerPermission (DRF)
Адміністратор (Admin)	Django Admin	Повний доступ через `/admin/`: блокування користувачів, модерація житла та відгуків, аналітика.	IsAdminUser / Staff

🛋️ 3. Інженерний пайплайн модуля 3D-візуалізації (3ds Max ➔ Web)

Ключова відмінність проєкту — демонстрація реалістичних інтер'єрів, створених у **Autodesk 3ds Max**:

1. 3D Моделювання
Геометрія кімнат, меблі, декор, матеріали та освітлення (Corona / V-Ray).

2. Рендеринг ракурсів
Генерація 3–6 ракурсів (вітальня, спальня, кухня, вид зверху) у 1920×1080.

3. WebP Оптимізація
Конвертація в WebP (< 300KB), збереження через модель Django `Property3DImage`.

4. Інтерактивна галерея
React-віджет з табами кімнат, зумом та бейджем "3D Інтер'єр" на фронтенді.

Карта REST API (DRF) та Нефункціональні вимоги

Маршрутизація ендпоінтів • Захист від Overbooking • Масштабованість

drf-spectacular (Swagger)

🗺️ 4. Карта ендпоінтів REST API (Django REST Framework)

МОДУЛЬ / МАРШРУТ	МЕТОД	ПРИЗНАЧЕННЯ ТА ПАРАМЕТРИ	ДОСТУП
`/api/auth/register/`	POST	Реєстрація користувача (email, пароль PBKDF2/bcrypt, ім'я, роль)	Публічний
`/api/auth/token/`	POST	Аутентифікація SimpleJWT, повернення access та refresh токенів	Публічний
`/api/properties/`	GET	Пошук: `city`, `check_in`, `check_out`, `guests`, `min_price`, `amenities`	Публічний
`/api/properties/<id>/`	GET	Деталі житла: опис, зручності, фото та **список 3D-рендерів**	Публічний
`/api/properties/<id>/images-3d/`	POST	Завантаження 3D-рендеру з полями `room_type` та `camera_angle`	Власник / Адмін
`/api/bookings/check-availability/`	POST	Перевірка зайнятості дат та попередній розрахунок вартості	Публічний / User
`/api/bookings/`	POST	Створення бронювання, транзакційне блокування дат, Mock Payment	Авторизований User
`/api/bookings/my/`	GET	Список бронювань користувача (активні, минулі, скасовані)	Авторизований User
`/api/bookings/<id>/cancel/`	PATCH	Скасування бронювання та миттєве звільнення дат у календарі	User / Власник
`/api/properties/<id>/reviews/`	POST	Створення відгуку з оцінкою 1–10 після завершення проживання	User (після заїзду)
`/admin/`	UI	Повнофункціональна панель **Django Admin** для модерації	Staff / Superuser

🛡️ 5. Захист від Overbooking та Бізнес-правила

Транзакційність (Django ORM + ACID)
Бронювання виконується всередині `transaction.atomic()` з використанням `select_for_update()`. Перевіряється перетин інтервалів `check_in < :new_out AND check_out > :new_in`. При конфлікті повертається `409 Conflict`.

Адаптивність та Швидкодія
Інтерфейс React оптимізовано під екрани від 375px (Mobile) до 1440px (Desktop). Для зображень використовується Lazy Loading, а для 3D-рендерів — формат WebP із кешуванням на клієнті.

⚙️ 6. Інфраструктура та середовище розгортання

Docker Compose сетап: контейнер `backend` (Python 3.11 / Django 5 + Gunicorn, порт 8000), контейнер `postgres` (PostgreSQL 16, порт 5432), контейнер `frontend` (Vite / Nginx, порт 5173/80). Swagger UI доступний за адресою `http://localhost:8000/api/docs/`.